cda_summary

Alberto Rueda

May 2026

1 Week 1

1. The Machine Learning Landscape

Learning Targets and Terminology

The goal of learning from data is to find an unobservable target function using a finite dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$. Machine learning methods are categorized based on the nature of the response variable y:

- **Unsupervised Learning**: No response variable is available. The focus is on finding structure within the data.
  - **Clustering**: Used for customer segmentation, targeted marketing, and recommended systems.
  - **Dimensionality Reduction**: Used for meaningful compression, big data visualization, and structure discovery (e.g., Principal Component Analysis, Feature Elicitation).
- **Supervised Learning**: Response variable(s) are available.
  - **Classification (Categorical Response)**: Used for image classification, fraud detection, diagnostics, and customer retention. Specific methods include Linear Discriminant Analysis (LDA), K-Nearest Neighbors (KNN), and Support Vector Machines (SVM).
  - **Regression (Continuous Response)**: Used for forecasting, predictions, and process optimization. Specific methods include Ordinary Least Squares (OLS), Ridge Regression, and KNN.
- **Reinforcement Learning**: Focuses on agents taking actions in an environment to maximize rewards. Used for real-time decisions, Game AI, robot navigation, and skill acquisition.

Predictor Names by Domain

- **Machine Learning**: X inputs, Y outputs.
- **Statistics**: X predictors, Y responses.
- **Pattern Recognition**: X features, Y responses.

2. Mathematical Foundations and Notation

Data Definitions

- **Data Matrix** X: A $(n \times m)$ matrix where rows correspond to n samples and columns correspond to m variables.
- **Response Variable** y: A $(n \times 1)$ vector representing the output.
- **Model Coefficients** β: A $(m \times 1)$ vector representing the learned parameters.
- **Model Error Vector** e: A $(n \times 1)$ vector representing inherent noise.
- **Prediction Error Vector** ϵ: A $(n \times 1)$ vector, also known as the residual.

The Linear Model

A regression model that is linear in β is expressed as:

$$y = X\beta + e$$

3. Performance and Error Metrics

Error Types

- **Training Error**: An optimistic estimate of performance based on the data used to train the model.
- **Test Error**: Performance on a "unseen world" or a separate test dataset, representing generalization capability.
- **Generalization Gap**: The difference between the Training Error and the Test Error.

Expected Prediction Error (EPE)

The EPE is the "Gold Standard" theoretical target. It averages over the entire population distribution $P(X, Y)$:

$$EPE(\hat{f}) = \mathbb{E}[(Y - \hat{f}(X))^2]$$

Irreducible Noise (The Oracle's Error)

Even a perfect "Oracle" model has a lower bound of error due to irreducible noise ϵ (randomness from measurement error or hidden variables):

$$\text{True Process: } Y = f(X) + \epsilon$$

$$\text{Irreducible Error: } \sigma^2 = Var(\epsilon)$$

$$\text{Total Error} = \text{Reducible Error} + \sigma^2$$

4. Bias-Variance Decomposition

The Expected Prediction Error at a specific point x_0 can be decomposed into three sources:

$$EPE(x_0) = \mathbb{E}_{y,D|x_0}||y(x_0) - \hat{f}(x_0; D)||^2$$

$$EPE(x_0) = \sigma_e^2 + \text{bias}^2(\hat{f}(x_0; D)) + \text{variance}(\hat{f}(x_0; D))$$

Where the estimate $\hat{f}(x_0)$ is based on observed $y(x_0) = f(x_0) + e$ (with $\mathbb{E}(e) = 0$ and $Var(e) = \sigma_e^2$) and training data D.

1. Irreducible Error

$$\sigma_e^2 = \mathbb{E}_y(y(x_0) - f(x_0))^2$$

2. Statistical Bias

The difference between the expected value of an estimate and the true value. It represents systematic error due to incorrect model assumptions.

$$\text{bias}^2(\hat{f}(x_0; D)) = (\mathbb{E}_D(\hat{f}(x_0; D)) - f(x_0))^2$$

$$Bias(\hat{\theta}) = \mathbb{E}(\hat{\theta}) - \theta$$

3. Variance

Quantifies how much the model estimate fluctuates if the experiment is repeated with different training sets. It represents sensitivity to small fluctuations in the data.

$$\text{variance}(\hat{f}(x_0; D)) = \mathbb{E}_D(\hat{f}(x_0; D) - \mathbb{E}_D(\hat{f}(x_0; D)))^2$$

$$Var(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}(\hat{\theta}))^2]$$

5. Linear Regression Models

Ordinary Least Squares (OLS)

Finds the β that minimizes the Residual Sum of Squares (RSS).

$$\text{Minimize: } ||y - X\beta||_2^2 = \sum_{i=1}^{n}(y_i - X_i\beta)^2$$

$$\hat{\beta}_{OLS} = \arg\min_{\beta} ||y - X\beta||_2^2$$

The closed-form solution is found by setting the gradient to zero:

$$\frac{\partial}{\partial\beta}(y - X\beta)^T(y - X\beta) = 0$$

$$-2X^T(y - X\beta) = 0$$

$$X^Ty - X^TX\beta = 0$$

$$\hat{\beta}_{OLS} = (X^TX)^{-1}X^Ty$$

Properties of OLS:

- **BLUE**: Best Linear Unbiased Estimator.
- **Unbiased**: $\mathbb{E}(\hat{\beta}_{OLS}) = \beta$.
- **Best Unbiased**: $Var(\hat{\beta}_{OLS}) \leq Var(\hat{\beta}_{linear})$.
- **Requirements**: X^TX must be invertible (non-singular). If features are highly correlated, the matrix is near-singular, causing high variance (instability).

Ridge Regression

Introduces bias deliberately to lower the variance of $\hat{y} = X\beta$ by shrinking the size of β.

$$\hat{\beta}_{ridge} = \arg\min_{\beta} ||y - X\beta||^2 + \lambda||\beta||^2$$

We introduce bias to lower variance

The closed-form solution stabilizes the inverse numerically by adding a "ridge" to the diagonal:

$$\hat{\beta}_{ridge} = (X^TX + \lambda I)^{-1}X^Ty$$

Implementation Requirements:

- **Centering**: Data must be centered (mean of each variable is zero) so shrinkage is toward zero.
- **Normalization**: Data must be scaled (standard deviation of 1) so the penalty term $||\beta||^2$ treats all variables with equal importance.

6. Model Complexity and Selection

Defining Complexity

Model complexity refers to the flexibility or Degrees of Freedom (df) of the estimator $\hat{f}$.

- **Parametric models**: Often tied to the number of parameters m.
- **Non-parametric models (e.g., KNN)**: Tied to neighborhood size or the strength of a penalty.

The Complexity Trade-off

- **Low Complexity**: High Bias (assumes too much), Low Variance (stable) $\rightarrow$ **Underfitting**.
- **High Complexity**: Low Bias (fits training data well), High Variance (unstable) $\rightarrow$ **Overfitting**.

The m vs. n Relationship

Complexity is relative to the amount of data n:

- **Stable Regime ($n \gg m$)**: Data constrains the model; training and test errors are close.
- **Overfitting Regime ($n \approx m$)**: Model fits noise; large generalization gap; instability explodes.

Controlling the Complexity 'Knob'

Method	Complexity Knob	High Complexity
OLS	Fixed (m)	Large m
Ridge	Regularization λ	Small $\lambda \to 0$

Finding the "Sweet Spot"

Optimal complexity is found where test performance is estimated to be best. Since EPE is unobservable, we use:

- **Validation Sets**
- **Cross-Validation**
- **Information Criteria**: (AIC/BIC)

Week 2

The K-Nearest Neighbors (KNN) Algorithm

- **Classification**: Classifies an observation based on the majority class of its K nearest neighbors.
- **Regression**: Estimates the response as the average response of the K nearest neighbors.
- **Distance Measure**: Typically uses Euclidean distance to define proximity.
- **Standardization**: It is general practice to standardize variables to mean zero and variance 1 so each feature contributes equally to the distance.
- **Complexity**: K is the tuning parameter. Small K yields low bias/high variance (flexible); large K yields high bias/low variance (simple).

2. Model Error and Complexity

Bias-Variance Decomposition

The Expected Prediction Error (EPE) at a point x_0 is:

$$EPE(x_0) = \sigma_e^2 + \text{bias}^2(\hat{f}(x_0)) + \text{variance}(\hat{f}(x_0))$$

- **Irreducible Error** (σ_e^2): The noise floor inherent in the system.
- **Statistical Bias**: Systematic error from incorrect model assumptions ($\mathbb{E}[\hat{\theta}] - \theta$).
- **Variance**: Error from sensitivity to small fluctuations in the training data ($\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$).

Overfitting vs. Underfitting

- **Underfitting (Too Simple)**: Data is not accurately described; model assumptions are wrong (High Bias).
- **Overfitting (Too Complex)**: The model fits the noise in the data; high sensitivity to training set fluctuations (High Variance).
- **Complexity Knob**: OLS complexity is fixed by the number of variables m. Ridge complexity is controlled by the regularization parameter λ.

3. Linear Regression and Regularization

Ordinary Least Squares (OLS)

Finds β to minimize the Residual Sum of Squares (RSS):

$$\hat{\beta}_{OLS} = \arg\min_{\beta} \|y - X\beta\|_2^2 = (X^T X)^{-1} X^T y$$

OLS is the Best Linear Unbiased Estimator (BLUE) but can be unstable if $X^T X$ is near-singular.

Ridge Regression (L_2 Penalty)

Deliberately adds bias to reduce variance by shrinking coefficients toward zero.

$$\hat{\beta}_{Ridge} = \arg\min_{\beta} \|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

The closed-form solution is:

No bias
High Variance

High bias
No Variance

$$\beta_{Ridge} = (X^T X + \lambda I)^{-1}(X^T Y)$$

$\lambda = 0$ results in OLS; $\lambda \to \infty$ results in $\beta \to 0$.

Lasso Regression (L_1 Penalty)

Used for variable selection by forcing some coefficients to exactly zero.

$$\hat{\beta}_{Lasso} = \arg\min_{\beta} \|Y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

Equivalently, the constrained problem (basis pursuit) is:

$$\min \|Y - X\beta\|_2^2 \quad \text{subject to} \quad \sum_j |\beta_j| \leq s$$

Elastic Net

Combines L_1 and L_2 penalties to handle high dimensionality ($p > n$) and the grouping effect where correlated predictors are selected or dropped together.

$$\min_{\beta} \frac{1}{2n}\|Y - X\beta\|_2^2 + \lambda\left(\frac{1-\alpha}{2}\|\beta\|_2^2 + \alpha\|\beta\|_1\right)$$

4. Model Selection and Assessment

Data Splitting and Cross-Validation

- **Training Set**: Used to build models with different tuning parameters.
- **Validation (Dev) Set**: Used to tune parameters and select the best model.
- **Test Set**: Used once to estimate the final performance on unseen data.
- **K-fold Cross-Validation**: Divides data into K parts. For each k, the model is fit on $K-1$ parts.

$$CV(\lambda) = \frac{1}{K}\sum_{k=1}^{K} Err_k(\lambda)$$

- **One Standard Error (1-SE) Rule**: Choose the simplest model within 1 SE of the minimum error to ensure stability.

$$S.E.(\lambda) = \frac{1}{\sqrt{K}}\sqrt{\frac{1}{K}\sum_{k=1}^{K}(Err_k(\lambda) - CV(\lambda))^2}$$

Information Criteria

Methods to estimate in-sample error without a separate validation set.

- **Optimism**: $op \equiv Err_{in} - \bar{err}$. In the linear case: $\mathbb{E}[Err_{in}] = \mathbb{E}[\bar{err}] + 2 \cdot \frac{d}{N}\sigma_e^2$.
- **C_p Statistic**: $C_p = \bar{err} + 2 \cdot \frac{d}{N}\hat{\sigma}_e^2$. penalty for model complexity. By choosing model with smallest CP, we choose the model with the best balance btw fit & simplicity
- **AIC (Akaike)**: $AIC = -\frac{2}{N}\log L + 2\frac{d}{N}$.
- **BIC (Bayesian)**: $BIC = -2\log L + \log(N)d$. BIC penalizes complexity more heavily than AIC as N grows.
- **Effective Parameters**: $df(S) = \text{trace}(S)$ where $\hat{Y} = SY$.

$C_p \approx$ training error + complexity penalty

AIC $\approx$ badness of fit + complexity penalty

Nested Cross-Validation

Used to solve "Selection-Induced Bias" where hyperparameters are overfitted to the validation set.

- **Inner Loop**: Selection/Tuning of λ.
- **Outer Loop**: Assessment/Audit of the entire "Selection + Training" pipeline.

Bootstrapping

A general method for assessing statistical accuracy by randomly drawing datasets with replacement from the original training data.

$$\widehat{Var}[S(Z)] = \frac{1}{B-1}\sum_{b=1}^{B}(S(Z^{*b}) - \bar{S}^*)^2$$

Week 3

1. High-Dimensional Inference and Algorithms

Curse of Dimensionality

As dimension D increases, the number of regions grows exponentially. Data becomes sparse, Euclidean distances lose meaning (points become equidistant), and models easily fit noise (overfitting).

LARS (Least Angle Regression)

A fast computational "engine" for finding the Lasso/Elastic Net path.

1. Start with all $\beta = 0$. Find x_j most correlated with y. pick the direction of its least-square coeff.
2. Move β_j until a second variable x_k is equally correlated with the current residual.
3. Move in an equiangular direction u_A bisecting the angle between variables in the active set A.

$$u_A = X_A w, \quad \text{where} \quad w = A(X_A^T X_A)^{-1}\mathbf{1}$$

Cyclical Coordinate Descent

Solves the Lasso iteratively by updating one coordinate β_j while holding others fixed.

1. Fix λ, solve $\min_\beta \frac{1}{2n}\sum_{i=1}^{n}(y_i - x_i\beta)^2 + \lambda|\beta|$ iteratively by
2. **Partial Residual**: $r_i^{(j)} = y_i - \sum_{k\neq j} x_{ik}\tilde{\beta}_k(\lambda)$.
3. **OLS for coordinate j**: $\tilde{\beta}_j^{OLS} = \frac{1}{n}\sum_i x_{ij} r_i^{(j)}$.
- **Soft Thresholding**: $\tilde{\beta}_j(\lambda) = \text{sign}(\tilde{\beta}_j^{OLS})(|\tilde{\beta}_j^{OLS}| - \lambda)_+$.

2. Performance Scorecard

Classification Metrics

- **Accuracy**: Total correct (dangerous for imbalanced data).
- **Sensitivity (Recall)**: Rate of correctly detecting the positive class.
- **Specificity**: Rate of correctly avoiding the negative class.
- **AUC-ROC**: General performance across all classification thresholds.

Regression Metrics

- **MSE/RMSE**: Outlier sensitive; useful for safety-critical audits. $MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$.
- **MAE**: Robust to outliers; direct physical interpretation. $MAE = \frac{1}{n}\sum|y_i - \hat{y}_i|$.
- R^2: Fraction of variance explained (relative measure).
- **Residual Plots**: Used for a "sanity check" to look for structural patterns.

3. Multiple Hypothesis Testing

M independent tests with significance level α

- **Family-Wise Error Rate (FWER)**: Probability of at least one false rejection. $FWER = 1 - (1-\alpha)^M$.
- **Bonferroni Correction**: Rescales α by the number of tests M. Reject if p-value $< \alpha/M$.
- **False Discovery Rate (FDR)**: Controls the expected proportion of false discoveries. $FDR = \mathbb{E}\left[\frac{FP}{FP+TP}\right]$.
- **Benjamini-Hochberg Algorithm**: Sort p-values $p_{(1)} \leq \ldots \leq p_{(m)}$. Find max k such that $p_{(k)} \leq \frac{k}{m}q$, then reject all $H_{(1)}, \ldots, H_{(k)}$. (for a given q)

Week 4

1. Generative Classifiers: LDA and QDA

Generative Approach via Bayes' Theorem

Instead of modeling decision boundaries, we model the class-conditional density $f_k(x) = P(X = x|G = k)$ and use class priors $\pi_k = P(G = k)$.

(Class-conditional density, Prior)

$$P(G = k|X = x) = \frac{f_k(x)\pi_k}{\sum_{l=1}^{K} f_l(x)\pi_l}$$

Multivariate Gaussian Assumption

The class-conditional density is assumed to be: (Gaussian)

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k)\right)$$

Linear Discriminant Analysis (LDA)

Assumption: All classes share the same covariance matrix ($\Sigma_k = \Sigma_l = \Sigma$) (but different means). **Log-Odds**: The log-ratio of posterior probabilities becomes linear because quadratic terms cancel:

$$\log\frac{P(G=k|X=x)}{P(G=l|X=x)} = \log\frac{\pi_k}{\pi_l} - \frac{1}{2}(\mu_k+\mu_l)^T\Sigma^{-1}(\mu_k-\mu_l) + x^T\Sigma^{-1}(\mu_k-\mu_l)$$

$= \log\frac{f_k(x)}{f_l(x)} + \log\frac{\pi_k}{\pi_l}$ → if { log-ratio > 0 => class k; log-ratio < 0 => class l }

Linear Discriminant Function:

To classify x, we calculate $\delta_k(x)$ for all classes and assign x to the class with higher score: $\hat{G}(x) = \arg\max_k \delta_k(x)$

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log\pi_k$$

Parameter Estimation: → We estimate them from the training data:

- $\hat{\pi}_k = N_k/N$
- $\hat{\mu}_k = \sum_{g_i=k} x_i/N_k$
- Pooled Covariance: $\hat{\Sigma} = \sum_{k=1}^{K} \sum_{g_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T/(N-K)$

Quadratic Discriminant Analysis (QDA)

Assumption: Classes have unequal covariances ($\Sigma_k \neq \Sigma_l$). **Result**: The discriminant function is quadratic, and decision boundaries are conic subsections. It requires estimating $O(p^2)$ parameters per class, increasing variance in high dimensions.

RRDA : • Projects data into a k-1 dimensional subspace that maximizes class separation
• Excellent for visualisation and dim reduction prior to classification.

Regularized Discriminant Analysis (RDA)

RDA shrinks the covariance matrices to improve stability:

Regularisation Options

- **Option 1**: Shrink QDA toward LDA: $\hat{\Sigma}_k(\alpha) = \alpha\hat{\Sigma}_k + (1-\alpha)\hat{\Sigma}$.
- **Option 2**: Shrink toward diagonal: $\hat{\Sigma}(\gamma) = \gamma\hat{\Sigma} + (1-\gamma)\text{diag}(\hat{\Sigma})$.
- **Option 3**: Shrink LDA toward spherical: $\hat{\Sigma}(\gamma) = \gamma\hat{\Sigma} + (1-\gamma)\hat{\sigma}^2 I$.

2. Discriminative Classifiers: Logistic Regression

The Discriminative Approach => Logistic - Regression

Logistic Regression models the posterior probabilities $P(G|X)$ directly without assuming Gaussian distributions for X. **Two-Class Case**: It focuses on specific features that separates the classes

$$P(Y=1|X=x) = \frac{e^{\beta_0+\beta^T x}}{1+e^{\beta_0+\beta^T x}}$$

$$P(Y=0|X=x) = \frac{1}{1+e^{\beta_0+\beta^T x}}$$

Log-Odds (Logit): $\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} = \beta_0 + \beta^T x$. → if { log-ratio > 0 => class 1 ; log-ratio < 0 => class 0

Fitting the Model: Maximum Likelihood (MLE)

Likelihood Function: $L(\beta_0, \beta) = \prod_{i=1}^{n} P(G = g_i|X = x_i)$. **Log-Likelihood**:

$$\ell(\beta) = \sum_{i=1}^{N} [y_i(\beta^T x_i) - \log(1 + e^{\beta^T x_i})]$$

} No closed form for MLE

This requires iterative solvers like Newton-Raphson to find β.

Multinomial Logistic Regression (Softmax) → Reference

For $K > 2$ classes, one class K is chosen as a pivot:

$$\log \frac{P(G=k|X=x)}{P(G=K|X=x)} = \beta_{k0} + x\beta_k$$

Recovering Probabilities:

$$P(G=k|X=x) = \frac{\exp(\beta_{k0} + x\beta_k)}{1 + \sum_{i=1}^{K-1} \exp(\beta_{i0} + x\beta_i)}$$

• Regularised Logistic Regression :
In cases of high dimension datasets, we can use elastic-net as regularisation for the likelihood.

3. Basis Expansions and Splines

Basis Expansion Principle → Features Transformation

Transform features X into a new space $h(X)$ to handle non-linearity using linear models:

$$y = \sum_{i=1}^{M} \beta_i' h_i(X)$$

Where to put knots: ○ where most data is ○ known threshold

Cubic Splines

at the knots

Divide the domain into regions separated by knots ξ. **Constraints**: Must be continuous, and have continuous 1st and 2nd derivatives at the knots. **Truncated Power Basis**:

$$f(t) = \beta_0 + \beta_1 t + \beta_2 t^2 + \beta_3 t^3 + \beta_4 (t - \xi_1)_+^3 + \dots$$

The "Hockey Stick" function $(x - \xi)_+^3$ is 0 if $x \leq \xi$ and $(x - \xi)^3$ if $x > \xi$.

Natural Cubic Splines

Adds the constraint that the function must be linear beyond the boundary knots, reducing erratic behavior at the tails and freeing up degrees of freedom.

Week 5

A good split separates the data into groups where the y-values are more similar within each group

1. Tree-Based Models: CART

How to build: 1. Start with the whole dataset 2. Try many possible splits and compute RSS for each 3. Choose split with lower RSS 4. Repeat in each child node and stop when nodes become too small 5. Predict in final node

Regression Trees

Partitions the space into regions and fits a constant in each. **Split Criterion**: Minimizes the Residual Sum of Squares (RSS).

Pruning Rule: Finding the right tree size
↳ grow the tree really large and then decide which splits were unnecessary (& remove them).
↳ Remove the ones that contribute less to lowering RSS

$$RSS_I = \sum_{i \in I} (y_i - \hat{y}_i)^2$$

Minimal when $\hat{y}$ is the average of observations in the interval.

We classify observation in nodes to class K st: $K = \underset{k}{\operatorname{argmax}}\ \hat{p}_k$

Classification Trees

Assigns the majority class of training observations in a node to new observations. **Node Impurity Measures** (p_{ik} is the proportion of class k in node i):

where $\hat{p}_k = \frac{1}{N} \sum_{x_i \in R} \mathbb{1}\{y_i = k\}$

- **Misclassification Error**: $Q = 1 - \hat{p}_{ik}$.
- **Gini Index**: $Q = \sum_k \hat{p}_{ik}(1 - \hat{p}_{ik})$.
- **Cross-entropy (Deviance)**: $Q = -\sum_k \hat{p}_{ik} \log(\hat{p}_{ik})$.

Tree Pruning

Grow the tree large, then prune nodes that contribute least to lowering RSS. **Weakest-Link Pruning**: Minimize $RSS(T) + \alpha|T|$, where $|T|$ is the number of terminal nodes and α is a tuning parameter found via cross-validation.

2. Ensemble Methods: Bagging

Bagging Algorithm

Bootstrap Aggregating reduces variance by averaging predictions from multiple models fitted to bootstrap samples.

1. Create B bootstrap samples of size N (sampling with replacement).
2. Fit a model to each sample to get prediction $\hat{y}_b$.
3. $\hat{y}_{\text{bagging}} = \frac{1}{B} \sum_{b=1}^{B} \hat{y}_b$.

Bagging Variance

If ρ is the correlation between pairs of trees and σ^2 is the variance of a single tree:

$$\text{Variance} = \rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2$$

As $B \to \infty$, the second term disappears, but the variance is still limited by the correlation ρ.

3. Appendix: Elastic Net Augmentation

The Augmentation Trick

One can solve the Elastic Net problem using a standard Lasso solver by augmenting the data. **Augmented Matrices**:

$$X_{aug} = \begin{pmatrix} X \\ \sqrt{\lambda_2} I_p \end{pmatrix}, \quad y_{aug} = \begin{pmatrix} y \\ 0_p \end{pmatrix}$$

Absorption:

$$\|y_{aug} - X_{aug}\beta\|_2^2 = \|y - X\beta\|_2^2 + \lambda_2 \|\beta\|_2^2$$

This absorbs the L_2 penalty into the error term.

Double Shrinkage Fix

To correct the bias where coefficients are "taxed" twice, a rescaling factor $c = (1 + \lambda_2)^{-1/2}$ is used:

$$X^* = \frac{1}{\sqrt{1 + \lambda_2}} \begin{pmatrix} X \\ \sqrt{\lambda_2} I_p \end{pmatrix}, \quad y^* = \begin{pmatrix} y \\ 0_p \end{pmatrix}$$

Rescaling ensures the OLS solution on this data matches the standard Ridge Regression formula.

Week 6

1. Ensemble Methods: Random Forests

Theory and Properties

- **Definition**: Random forest is a refinement of bagged trees that tries to improve performance by decorrelating the individual trees, thereby reducing overall variance.
- **Parallelization**: Because trees are grown independently, the code is easily parallelized.
- **Tuning**: Both Random Forests and Boosting are considered simple to tune and train.
- **Connection to Ridge**: The ensemble averaging reduces the contribution of any single variable, functioning similarly to shrinkage in Ridge regression. This makes it suitable for $n \ll p$ problems.

Random Forest Algorithm

1. **Initialization**: Define the number of trees, B. Typically a few hundred (overfitting is generally not a problem with B).
2. **Iteration**: For $b = 1$ to B:
   (a) Take a bootstrap sample of size N with replacement from the training data.
   (b) **Recursive Partitioning**: Repeat until a minimum node size n_{min} is reached (do not prune):
   - Take a random sample without replacement of the predictors (size $m < p$). → Bootstrap
   - Construct the CART partition by picking the best split among the m selected variables.
3. **Output**: A collection of B trees.

The Variance Reduction Formula

The variance of the average of bagged/forest estimates is defined by (where ρ is the correlation between pairs of trees and σ^2 is the variance of a single tree):

$$\text{Variance} = \rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2$$

- The second term $\to 0$ as B increases.
- The limiting factor is ρ; Random Forests use a random subset of variables at each split to lower ρ and decorrelate the trees.

Prediction and Model Selection

- **Classification**: Majority vote of the B trees.
- **Regression**: Arithmetic average of predictions: $\hat{y} = \frac{1}{B}\sum_{i=1}^{B} T_b(x)$.
- **Out-of-Bag (OOB) Estimates**: Samples not included in a bootstrap sample (OOB) are used to assess the performance of the corresponding tree.
- **OOB Stop Criterion**: Construct trees in sequence; stop when the OOB prediction error no longer decreases.
- **Parameter Heuristics**:
  - Classification: $m = \lfloor\sqrt{p}\rfloor$
  - Regression: $m = \lfloor p/3 \rfloor$

Variable Importance Measures

- **Gini Index**: The total improvement in the split-criterion for a specific variable is accumulated over all splits in all trees.
- **OOB Randomization**: For each tree, prediction accuracy is measured on OOB samples. The values of variable j are then permuted, and accuracy is re-measured. The average difference in accuracy across all trees indicates importance.

Proximity Matrix

- An $n \times n$ matrix where the (i, j) entry increases by one every time OOB samples i and j end up in the same terminal node.
- **Visualization**: Multidimensional scaling is used to represent the matrix in 2D. Points far from decision boundaries appear at the extremities (star-like), while points near boundaries are at the center.

2. Ensemble Methods: Boosting

Fundamentals

- **Concept**: Average many weak learners (e.g., small trees or "stubs") grown adaptively to reweighted versions of the data.
- **Bias-Variance**: Unlike Bagging (which only reduces variance), Boosting reduces **both** bias and variance.
- **Weights**: Observations are weighted; weights are updated to emphasize previous misclassifications.
- **Weak Learners**: Typically uses high-bias/low-variance shallow trees (stumps = single split).

AdaBoost.M1 Algorithm (Two-Class)

1. Initialize weights $w_i = 1/N$ for $i = 1, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit $G_m(x)$ to training data using weights w_i.
   (b) Compute weighted error: $err_m = \frac{\sum_{i=1}^{N} w_i \mathbb{I}(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}$.
   (c) Compute $\alpha_m = \log((1 - err_m)/err_m)$.
   (d) Update weights: $w_i \leftarrow w_i \exp[\alpha_m \mathbb{I}(y_i \neq G_m(x_i))]$ and renormalize so $\sum w_i = 1$.
3. Output final classifier: $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

Additive Models and Loss Functions

- **Basis Expansion**: Boosting can be viewed as $F(x) = \sum_{m=1}^{M} \beta_m b(x; \gamma_m)$, where $b(x; \gamma_m)$ is a tree.
- **Stagewise Fitting**: Parameters are optimized one at a time while holding previous trees fixed.
- **Exponential Loss**: $L(y, F(x)) = \exp(-yF(x))$. This loss is minimized at $F(x) = \frac{1}{2} \log \frac{P(y=1|x)}{P(y=-1|x)}$.
- **Relation to Logistic Regression**: $P(y = 1|x) = \frac{e^{2F(x)}}{1+e^{2F(x)}}$.
- **Noise Sensitivity**: Boosting is sensitive to mislabeled observations due to the exponential penalty on negative margins.

Gradient Tree Boosting

1. Initialize $F_0(x) = \arg\min_\gamma \sum L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) Compute negative gradient (residuals): $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$.
   (b) Fit regression tree to r_{im} giving terminal regions $R_{jm}, j = 1, \ldots, J_m$.
   (c) Compute terminal node values: $\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$.
   (d) Update model: $F_m(x) = F_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{I}(x \in R_{jm})$.
3. Output $G(x) = F_M(x)$.

Multiclass Boosting

- Loss function: $-\sum_{k=1}^{K} y_k \log(P(y_k|x))$.
- Probability from scores F_k: $P_k(y = 1|x) = \frac{e^{F_k(x)}}{\sum_{l=1}^{K} e^{F_l(x)}}$.
- Gradient of loss: $-g_k(x_i) = y_i - P_k(x_i)$.

Week 7

Tries to find the best boundary that separates classes while leaving the largest possible safety margin btw them

1. Support Vector Machines (SVM)

EX: it tries to draw a line separating groups.
- 2D: the boundary is a line
- 3D: the boundary is a plane
- Higher dimensions: Hyperplane

But we want the line with the largest margin

Distance btw decision boundary and the closest data points for each class

The Linear Decision Function

Class labels are $y \in \{1, -1\}$. The decision function is $y_{new} = \text{sign}(\beta_0 + x_{new}\beta)$.

We want OSH (Optimal Separating Hyperplane)

Geometric Foundations

For OSH

- **Unit Normal**: $\hat{n} = \frac{\beta}{\|\beta\|}$ gives the shortest direction to the plane.
- **Point-to-Plane Distance**: $d = \frac{x_i^T \beta + \beta_0}{\|\beta\|}$. → Actual geometry distance
- **Margin**: SVM maximizes the signed distance $y_i \frac{x_i^T \beta + \beta_0}{\|\beta\|}$.

We can write: $\underset{\beta, \beta_0}{\text{Argmax}}\ C$ such that $y_i \frac{x_i\beta + \beta_0}{\|\beta\|} \geq C \ \forall i$

Optimization Problem (Primal)

- **Canonical Hyperplane**: Scale β, β_0 such that $|x_i^T \beta + \beta_0| = 1$ for the nearest points (Support Vectors).
- **Margin width**: $C = 1/\|\beta\|$. Maximizing C is equivalent to minimizing $\|\beta\|$.
- **Primal Objective**:

$$\arg\min_{\beta, \beta_0} \frac{1}{2}\|\beta\|^2 \quad \text{subject to} \quad y_i(x_i^T \beta + \beta_0) \geq 1 \quad \forall i$$

Lagrange Multipliers and Dual Problem

- **Lagrange Primal**: $L_p = \frac{1}{2}\|\beta\|^2 - \sum \alpha_i[y_i(x_i^T\beta + \beta_0) - 1]$, where $\alpha_i \geq 0$.
- **KKT Conditions**:
  1. Stationarity: $\beta = \sum \alpha_i y_i x_i$ and $\sum \alpha_i y_i = 0$.
  2. Complementary Slackness: $\alpha_i[y_i(x_i^T\beta + \beta_0) - 1] = 0$.
- **Dual Objective**:
$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j \langle x_i, x_j\rangle$$
subject to $\alpha_i \geq 0$ and $\sum \alpha_i y_i = 0$.

Kernels and Non-Linearity

- **The Kernel Trick**: Replace the dot product $\langle x_i, x_j\rangle$ with a kernel function $K(x_i, x_j)$.
- **Polynomial Kernel**: $K_{i,j} = (1 + x_i^T x_j)^d$.
- **Radial/Gaussian Kernel**: $K_{i,j} = \exp(-\gamma\|x_i - x_j\|^2)$. This represents a dot product in an infinite-dimensional space.
- **Neural Network Kernel**: $K_{i,j} = \tanh(c_1 x_i^T x_j + c_2)$.

The Support Vector Machine (Overlapping Classes)

- **Slack Variables**: Allow points to be on the wrong side of the margin by introducing $\xi_i \geq 0$.
- **Soft Margin Primal**:
$$\arg\min_{\beta,\beta_0} \frac{1}{2}\|\beta\|^2 + \lambda\sum \xi_i \quad \text{s.t.} \quad y_i(x_i^T\beta + \beta_0) \geq 1 - \xi_i$$
- **Soft Margin Dual**:
$$\arg\max_{\alpha} \sum \alpha_i - \frac{1}{2}\alpha^T\mathbf{YKY}\alpha \quad \text{s.t.} \quad 0 \leq \alpha_i \leq \lambda, \quad \sum \alpha_i y_i = 0$$

Key Properties and Caveats

- **Sparsity**: The model is defined only by Support Vectors ($\alpha_i > 0$). You could delete 90% of "safe" points and the boundary wouldn't move.
- **Scalability**: Generally limited to 10,000–20,000 observations.
- **Variable Selection**: Does not perform automatic variable selection.
- **Multiclass**: Does not generalize naturally to > 2 classes (though frequently used regardless).
- **Extensions**: Support Vector Regression and One-Class SVM (for anomaly detection).

Week 8

Subspace Models

The Dimensionality Problem and High-Dimensional Data

In modern applications, data is often highly dimensional ($p \gg n$). For example, audio clips can yield thousands of features, exceeding the number of observations.

- **The Curse of Dimensionality (Volume Explosion)**: In 1-D, 100 points occupy 75% of the space. In 10-D, 100^{10} points are needed for the same coverage.

- **The Manifold Hypothesis**: High-dimensional data is rarely truly p-dimensional; it typically lives on a low-dimensional manifold.
- **Subspace Solutions**: Dimensionality Reduction techniques.
  - Unsupervised compression: PCA / SVD.
  - Interpretable extraction: Sparse PCA.
  - Supervised projection: Partial Least Squares (PLS).
  - Multi-modal alignment: Canonical Correlation Analysis (CCA).

Principal Component Analysis (PCA) → looks for structure in the data; does not confirm pre-made hypothesis or predict an outcome

PCA is an exploratory unsupervised method used to find structure, detect outliers, and reduce dimensionality. It identifies the axes through a data cloud that capture the widest possible spread.

*Preprocessing and Objective 1. **Center the Data**: Remove the mean from raw observations Z: $X = Z - \mu_Z$. 2. **Linear Transformation**: $S = XL$, where L is a rotation matrix such that $L^T L = I$. 3. **Maximize Variance**: We aim to maximize the variance of the projected data columns (Principal Components).

$$\text{cov}(S) = \frac{1}{n} S^T S = \frac{1}{n} L^T X^T X L = L^T \Sigma L$$

Where $\Sigma = \text{cov}(X)$. The first PC objective is:

$$\arg\max_l l^T \Sigma l \quad \text{subject to} \quad l^T l = 1$$

*The Lagrangian and Eigenvalue Problem To solve the constrained optimization, we construct the Lagrangian:

$$L_p = l^T \Sigma l - \lambda(l^T l - 1)$$

Taking the partial derivative and setting it to zero:

$$\frac{\partial L_p}{\partial l} = 2\Sigma l - 2\lambda l = 0 \implies \Sigma l = \lambda l$$

- **Mathematical Conclusion**: Covariance is maximized when l is an eigenvector of Σ. The maximal variance achieved is the corresponding eigenvalue λ.
- **Orthogonality**: Because Σ is symmetric, eigenvectors are automatically orthogonal, forming uncorrelated PC axes.

*Computation via Singular Value Decomposition (SVD) For high-dimensional data ($p > n$), computing $\Sigma = X^T X$ is computationally expensive and numerically unstable. SVD is preferred:

$$X = UDV^T$$

Where U are left singular vectors, D are diagonal singular values, and V are right singular vectors.

- **Loadings (L)**: Equal to V. They act as a "recipe" for building PCs from original features.
- **Scores (S)**: The coordinates of data points on the new axes ($S = XL$).
- **Variance Connection**: $X^T X = VD^2V^T$. Singular values are the square roots of scaled eigenvalues: $S_d = \sqrt{nD}$.

*Model Selection and Number of Components

- **Explained Variance**: $100 \times \frac{\lambda_i}{\sum \lambda_j}$.
- **Kaiser Criterion**: Keep eigenvectors with eigenvalues $\lambda > 1$.
- **Scree Plot**: Compare eigenvalues to randomized data eigenvalues.

Aims for a sweet-spot btw { Maximizing Variance & Force weights to 0 } => We maximize variance, keeping loadings independent & forcing irrelevant variable weights to exactly 0.

Sparse PCA (SPCA)

Standard PCA loadings are dense, using every variable. Sparse PCA aims to drive irrelevant coefficients to zero for better interpretability.

- **Thresholding Trap**: Arbitrarily zeroing small loadings (e.g., $|l_j| < 0.15$) is a bad idea because coefficients are interdependent. It destroys orthogonality ($L^T L \neq I$) and makes scores correlated ($S^T S$ is no longer diagonal).
- **Varimax Criterion**: Rotates principal components to maximize the variance among loadings, producing approximate sparseness.
- **Elastic Net Approach**: Expresses each PC as a regression problem with penalties:

$$\arg\min_{l} ||x_i - Xl||^2 + \lambda||l||_2^2 + \gamma||l||_1$$

Partial Least Squares (PLS)

Finds components that explain variation in X (as PCA), while also being useful for predicting outcome y.

PLS is a supervised method that seeks directions having both high variance and high correlation with a response variable y.

- **Objective**: The m-th PLS direction φ_m solves:

$$\max_{\alpha} \text{Corr}^2(y, X\alpha)\text{Var}(X\alpha) \quad \text{s.t.} \quad ||\alpha|| = 1, \alpha^T \Sigma \varphi_l = 0$$

- **Iterative Algorithm**:
  1. Standardize X and y.
  2. Weights: $\hat{\varphi}_{mj} = (x_j^{(m-1)})^T y$.
  3. Latent Component: $z_m = \sum \hat{\varphi}_{mj} x_j^{(m-1)}$.
  4. Regression Coefficient: $\hat{\theta}_m = \frac{z_m^T y}{z_m^T z_m}$.
  5. Update Prediction: $\hat{y}^{(m)} = \hat{y}^{(m-1)} + \hat{\theta}_m z_m$.
  6. Deflation: $x_j^{(m)} = x_j^{(m-1)} - \left(\frac{z_m^T x_j^{(m-1)}}{z_m^T z_m} \right) z_m$.
- **Properties**: Components are uncorrelated. If $M = p$, PLS is equivalent to OLS.

Canonical Correlation Analysis (CCA)

→ Method to study the relationships btw two sets of variables

CCA finds associations between two distinct data groups (X and Y) of the same subjects (multi-modal alignment).

- **Objective**: Focuses on cross-correlation, ignoring internal variance.

$$\max_{u,v} \frac{u^T \Sigma_{XY} v}{\sqrt{u^T \Sigma_{XX} u \cdot v^T \Sigma_{YY} v}}$$

- **Generalized Eigenvalue Problem**:

$$\Sigma_{XY} \Sigma_{YY}^{-1} \Sigma_{YX} u = \lambda^2 \Sigma_{XX} u$$

- **Regularization**: Standard CCA fails if $p > n$ because covariance matrices become singular. Sparse CCA (using L_1 penalties) or Ridge-regularized CCA solves this.

Week 9

Cluster Analysis

Definitions and Goals

Clustering is unsupervised classification. Given a set of points, it partitions them into clusters such that points in the same cluster are "close" while points in different clusters are "far apart." It is used for seeing structure, dimensionality reduction, and outlier detection.

Distance and Dissimilarity Measures

- **Euclidean distance**:
$$d(x_i, x_j) = \sqrt{\sum_{k=1}^{p} (x_{ik} - x_{jk})^2}$$
- **Manhattan (City Block/Hamming) distance**:
$$d(x_i, x_j) = \sum_{k=1}^{p} |x_{ik} - x_{jk}|$$
- **Mahalanobis distance**: Accounts for equal dispersion Σ. → Points are closer if their difference lies in a direction where the data naturally vary a lot
$$d(x_i, x_j) = \sqrt{(x_i - x_j)^T \Sigma^{-1} (x_i - x_j)}$$
- **Tanimoto distance**: Used for categorical/binary vectors. (Jaccard for binary classification)
$$d(x_i, x_j) = \frac{x_i^T x_j}{x_i^T x_i + x_j^T x_j - x_i^T x_j}$$
- **Weighted distances**: $d(x_i, x_j) = \sum_{k=1}^{p} w_k d_k(x_{ik}, x_{jk})$ where $\sum w_k = 1$.

K-means and K-medoids

- **K-means Algorithm**:
  1. Randomly initialize K centroids.
  2. Assign each point to the closest centroid.
  3. Update centroids as the mean of assigned points.
  4. Repeat until assignments do not change.
- **Properties**: Sensitive to initialization. → Results depend on initialization
- **K-medoids (PAM)**: Uses actual observations as cluster centers. It is more robust to outliers but computationally heavier than K-means.

Hierarchical Clustering

Generates a tree (dendrogram).

- **Approaches**:
  - **Agglomerative (Bottom-up)**: Start with individuals, combine into bigger clusters.
  - **Divisive (Top-down)**: Start with one cluster, divide into smaller ones.
- **Linkage Measures** (distance between clusters G and H):
  - **Complete (Furthest-neighbor)**: $d_{CL} = \max_{i \in G, j \in H} d_{ij}$. Tends to produce balanced trees.
  - **Single (Nearest-neighbor)**: $d_{SL} = \min_{i \in G, j \in H} d_{ij}$. Can suffer from "chaining" and unequal cluster sizes.
  - **Ward-linkage**: Measures the increment in within-cluster sum of squares. Only used with Euclidean distance.
$$d_{Ward} = \sqrt{\frac{n_G n_H}{n_G + n_H} ||\bar{x}_G - \bar{x}_H||_2^2}$$
- **Biclustering**: Simultaneous clustering of both samples and variables (e.g., genes and patients), typically visualized as a heatmap.

Gaussian Mixture Models (GMM) and EM Algorithm

GMM provides probabilistic "soft" cluster assignments. Data is assumed to belong to one of several Gaussian distributions selected by a latent variable Z.

- **Joint Log-Likelihood**:

$$\ell(\theta; x, Z) = \sum_{i=1}^{n} \sum_{j=1}^{K} \mathbb{I}_{\{Z_i=j\}} (\log N(x_i; \mu_j, \Sigma_j) + \log \pi_j)$$

- **EM Algorithm**:

1. **Expectation Step**: Compute posterior probabilities γ_{ij} using Bayes' formula:

$$\gamma_{ij} = p_{\theta^{(k)}}(Z_i = j | x_i) = \frac{\pi_j^{(k)} N(x_i; \mu_j^{(k)}, \Sigma_j^{(k)})}{\sum \pi_l^{(k)} N(x_i; \mu_l^{(k)}, \Sigma_l^{(k)})}$$

2. **Maximization Step**: Calculate new parameters:

$$\mu_j^{(k+1)} = \frac{\sum \gamma_{ij} x_i}{\sum \gamma_{ij}}, \quad \Sigma_j^{(k+1)} = \frac{\sum \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^T}{\sum \gamma_{ij}}, \quad \pi_j^{(k+1)} = \frac{1}{n} \sum \gamma_{ij}$$

Model Validation

How to select the number of clusters K:

- **Silhouette Method**: Measures cluster appropriateness for observation i.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

  Where $a(i)$ is average distance to points in the same cluster and $b(i)$ is average distance to the nearest neighboring cluster. Choose K that maximizes average silhouette.

- **Gap-statistic**: Compares log within-cluster dissimilarity W_k to expected dissimilarity U_k from a uniform distribution.

$$G(K) = \log(U_k) - \log(W_k)$$

- **Other criteria**: AIC, BIC, Chi-squared, Kolmogorov-Smirnov.

Other Clustering Methods

- **Louvain/Leiden**: Network-based community detection.
- **HDBSCAN**: Density-based.
- **Deep Clustering**: Using neural networks.

Week 10

Artificial Neural Networks (ANN)

The Four Ingredients of Deep Learning

To build and train a modern Deep Learning model, four primary ingredients are required:

- **Data (D)**: Observed features X and targets Y.
- **The Objective (L)**: A loss function to minimize, usually derived from the Negative Log-Likelihood (NLL).
- **The Engine (∇)**: An optimizer (typically Gradient Descent) and an algorithm to find gradients (Backpropagation).
- **Architecture (f_θ)**: The choice of non-linear function approximator, such as Multi-Layer Perceptrons (MLPs), Convolutional Neural Networks (CNNs), or Transformers.

Optimization Algorithm: Gradient Descent

Gradient Descent is used to minimize a loss function $L(w)$ by iteratively moving in the direction of steepest descent.

1. **Initialize**: Choose an initial guess for the parameters w.
2. **Compute Gradient**: Calculate the gradient of the objective function $L(w)$ with respect to the parameters, denoted as $\nabla L(w)$.
3. **Update Parameters:** Update the parameters in the direction opposite to the gradient:
$$w = w - \eta \cdot \nabla L(w)$$
where η is the learning rate, controlling the size of the steps.
4. **Convergence Criteria**: Stop when the change in parameters is sufficiently small or after a fixed number of iterations.

Likelihood and Loss Functions

Minimizing the Negative Log-Likelihood is the standard way to optimize Deep Learning models.

$$w^* = \arg\max_w \ell(D; w) \iff w^* = \arg\min_w -\ln \ell(D; w)$$

*Deriving MSE from Gaussian Likelihood Assume model output is the mean of a Gaussian distribution: $y_i = h_w(x_i) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. The Negative Log-Likelihood for the dataset is:

$$-\ln \mathcal{L}(w) = N \ln(\sqrt{2\pi\sigma^2}) + \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - h_w(x_i))^2$$

Minimizing this under a Gaussian assumption is equivalent to minimizing the **Mean Squared Error (MSE)**.

*Binary Cross-Entropy (BCE) Loss For classification with binary labels $y_i \in \{0, 1\}$, the BCE loss is:

$$L(w) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(w^T x_i)$. This is derived from the **Bernoulli likelihood**: $p(y_i|x_i, w) = \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$.

Backpropagation and Automatic Differentiation

Backpropagation is recursive calculus applied to matrices. Signal goes forward; "blame" for error goes backward.

*The Three Phases

1. **Forward Pass**: Input x flows through layers. Pre-activations $z^{(\ell)} = w^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$ and activations $a^{(\ell)} = \sigma(z^{(\ell)})$ are computed until reaching output $\hat{y}$.
2. **Backward Pass**: The Chain Rule is used to compute the error signal $\delta^{(\ell)}$ for each layer.
3. **Parameter Update**: Weights are nudged in the opposite direction of the gradient: $w \leftarrow w - \eta \cdot \nabla_w L$.

*Mathematical Formulas for Backprop The local derivative chain for a weight $w_{ij}^{(\ell)}$:

$$\frac{\partial L}{\partial w_{ij}^{(\ell)}} = \frac{\partial L}{\partial a_i^{(\ell)}} \cdot \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}} \cdot \frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}}$$

- **Error Signal** $\delta_i^{(\ell)} = \frac{\partial L}{\partial a_i^{(\ell)}} \cdot \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}}$
- **Vectorized Backprop Step**: $\delta^{(\ell)} = ((W^{(\ell+1)})^T \delta^{(\ell+1)}) \odot \sigma'(z^{(\ell)})$
- **Weight Update**: $\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} \times (a^{(\ell-1)})^T$

*Sigmoid Property The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ has the derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

This allows gradient calculation using only subtractions and multiplications if the output is known.

Deep Learning Architectures

*Convolutional Neural Networks (CNN) Designed with **Spatial Inductive Bias** for image-like data.

- **Local Connectivity**: Neurons only look at a small patch (receptive field).
- **Shared Weights**: Kernels/filters slide across the entire input, drastically reducing parameters.
- **Pooling Layers**: Downsamples feature maps (e.g., Max Pooling) to reduce dimensionality.
- **Translation Invariance**: Recognizes patterns regardless of position in the frame.

*Recurrent Neural Networks (RNN) Designed for sequential dependencies (time-series, text, audio).

- **Recurrence**: Hidden state h_t is computed using current input x_t and previous state h_{t-1}.
- **Parameter Sharing**: The same weights are used at every time step.

*Autoencoders Unsupervised models that learn a **Latent Bottleneck**.

- **Bottleneck**: A hidden layer smaller than the input forces the model to learn salient features.
- **Reconstruction Loss**: Minimizes $L(x, \hat{x})$, often via MSE.

*Transformers Uses the **Attention Mechanism** instead of recursion.

- **Self-Attention**: Allows every token to weigh the importance of every other token simultaneously.
- **No Recursion**: Computes all positions in parallel, speeding up training on GPUs.
- **Positional Encodings**: Injects "order" information directly into embeddings.

Week 11

Reveals hidden structures, latent variables or meaningful representations of the original data

Decomposition Methods for Unsupervised Learning

The core concept is approximating a data matrix X as the product of lower-dimensional factors: $X \approx WH$. (W: Basis Matrix, H: Activation Matrix)

1. Non-negative Matrix Factorization (NMF)

Constraint: All elements $w_{ik} \geq 0$ and $h_{kj} \geq 0$. → we ask for non-negativity because in many fields negative values are not realistic (e.g. MR, imaging, ...)

- **Parts-Based Representation**: Because features can only be added (not subtracted), the model learns localized "parts" (e.g., noses on faces) rather than holistic averages.
- **Objective Function (Squared Frobenius Norm)**:

$$\min_{W,H \geq 0} \frac{1}{2}\|X - WH\|_F^2 = \frac{1}{2}\sum_{i,j}(x_{ij} - \sum_k w_{ik}h_{kj})^2$$

- **Multiplicative Update Rules**:

$$H_{kj} \leftarrow H_{kj}\frac{(W^T X)_{kj}}{(W^T WH)_{kj}}, \quad W_{ik} \leftarrow W_{ik}\frac{(XH^T)_{ik}}{(WHH^T)_{ik}}$$

These are rescaled versions of gradient descent that maintain non-negativity.

2. Independent Component Analysis (ICA)

→ Example: Two people are talking with 2 microphones, we aim to separate their voices in the recordings

The goal is **Blind Source Separation**: Find un-mixing matrix $W \approx A^{-1}$ such that $\hat{s} = Wx$.

- **Assumptions**: Components are statistically independent and non-Gaussian.
- **Non-Gaussianity**: Measured by **Excess Kurtosis** $= \frac{\mu_4}{\sigma^4} - 3$.
- **Whitening**: Before optimization, data is centered and uncorrelated ($E[\tilde{x}\tilde{x}^T] = I$).
- **FastICA Rule**: (Uses TCL)

$$w_{new} \leftarrow E[\tilde{x}g(w^T\tilde{x})] - E[g'(w^T\tilde{x})]w$$

followed by normalization $w \leftarrow w/\|w\|$.

3. Archetypal Analysis (AA)

Focuses on the extremes (prototypes) that define the boundaries (convex hull) of the dataset.

- **Formula**: $X \approx XSH$ (where $Z = XS$ are archetypes).
- **Constraints**: $s_{ij} \geq 0, \sum_i |s_{ij}| = 1$ and $h_{ij} \geq 0, \sum_i |h_{ij}| = 1$.
- Archetypes are convex combinations of real data points; reconstructed data is a convex combination of archetypes.

4. Sparse Coding

Finding an overcomplete basis ($K > I$) where data is represented by few active components.

- **Objective Function**:

$$L(W, H) = \frac{1}{2}\|X - WH\|_F^2 + \lambda \sum_j \|h_j\|_1$$

- The L_1 penalty pushes small coefficients to exactly zero (Lasso problem).

Unsupervised Model Selection

'Speckled' Cross-Validation (Matrix Masking):

1. Randomly hide a percentage of entries in X.
2. Fit the model for k components while ignoring masked data.
3. Reconstruct the entire matrix.
4. Evaluate MSE only on masked entries: CV Error $= \sum_{\text{masked } i,j} (X_{ij} - \hat{X}_{ij})^2$.

Week 12

Multiway Models (Tensors)

Tensor Mathematics

A tensor is an N-dimensional array.

- **Frobenius Norm**: $\|\mathcal{X}\|_F = \sqrt{\sum_i \sum_j \sum_k x_{ijk}^2}$.
- **N-mode Multiplication**: Multiply an order N tensor by a matrix M along mode n:

$$[\mathcal{X} \times_n M]_{(n)} = M X_{(n)}$$

This involves unfolding the tensor into a matrix, performing multiplication, and folding it back.

Tucker3 Model

Decomposes a 3rd order tensor into a core tensor $\mathcal{G}$ and loading matrices A, B, C.

$$\mathcal{X} \approx \mathcal{G} \times_1 A \times_2 B \times_3 C = \sum_{p=1}^{P} \sum_{q=1}^{Q} \sum_{r=1}^{R} g_{pqr} a_p \circ b_q \circ c_r$$

- **Optimization (ALS)**:

$$A \leftarrow X_{(1)} (G_{(1)} (C \otimes B)^T)^\dagger$$

where † is the Moore-Penrose inverse.

- Good for data compression; not unique (can be adjusted by any rotation matrix Q).

Special case of Tucker (Core tensor $\mathcal{G}$ is super-diagonal)

Good for resolving additive physical profiles

PARAFAC (Parallel Factor Analysis)

Component 1 in mode A can only interact with components 1 in modes B & C
$a_1 \leftrightarrow b_1 \leftrightarrow c_1$
$a_2 \leftrightarrow b_2 \leftrightarrow c_2$

A special case of Tucker where the core tensor $\mathcal{G}$ is super-diagonal (identity-like).

$$\mathcal{X} \approx \sum_{r=1}^{R} a_r \circ b_r \circ c_r$$

- **Optimization (ALS)**:

$$A \leftarrow X_{(1)}(C \odot B)(C^T C * B^T B)^{-1}$$

  where $\odot$ is the Khatri-Rao product and $*$ is the Hadamard product.

- **Uniqueness**: PARAFAC is unique without additional constraints.

Model Selection for Tensors

CORCONDIA: High value: plausible structure; low value: model has too many components

- **CORCONDIA (Core Consistency Diagnostic)**: A heuristic to find the right number of components.

For a PARAFAC model with R components we obtain

$$\text{CORCONDIA} = 100 \cdot \left(1 - \frac{\|I - \mathcal{G}\|_F^2}{\|I\|_F^2}\right)$$

$I^{R \times R \times R}$ is a perfect super diagonal core tensor

- **Split-half Analysis**: Evaluate stability via the **Factor Match Score (FMS)**:
  - Split data into 2 halves, along one mode (usually sample mode)
  - Fit model separately for each half
  - Compare FMS evaluated on both halves.

$$\text{FMS} = \sum_{r=1}^{R} \frac{a_r^T \hat{a}_r}{\|a_r\|\|\hat{a}_r\|} \cdot \frac{b_r^T \hat{b}_r}{\|b_r\|\|\hat{b}_r\|} \cdot \frac{c_r^T \hat{c}_r}{\|c_r\|\|\hat{c}_r\|}$$

→ If the same factors are recovered from two subsets of the data, the chosen number of components is more trustworthy